Evaluation, Checkpointing, and Training Diagnostics

pig7selene

September 5, 2026

Abstract

Pretraining is an extended stochastic computation rather than a single optimization step. This chapter separates the losses that measure its progress from the signals that diagnose its execution, then develops a checkpoint as a resumable training state rather than a collection of model weights. It explains held-out validation, perplexity, online health metrics, loss spikes, failure localization, checkpoint cadence, sharded recovery, checkpoint selection, and experiment tracking. The central requirement is continuity: a monitor or restart is useful only when its counters, data identity, numerical state, and optimizer semantics remain explicit.

1 Introduction

Chapters 5 through 10 described the pretraining objective, data pipeline, optimizer, numerical controls, scale accounting, and distributed execution needed to update a language model. A long training run still needs an answer to a different question: how can its operators tell whether those components are producing the intended trajectory? The scalar loss is necessary evidence, but it is not a complete diagnosis. A training run can have a plausible loss while silently consuming an incorrect token stream, advancing its scheduler on the wrong clock, accumulating corrupted optimizer state, or failing to write a recoverable checkpoint.

Monitoring, evaluation, and checkpointing form one control loop. Monitoring records observations from the current step. Evaluation estimates performance on a protected distribution. A checkpoint preserves enough state to repeat the next update after interruption. The loop should detect deviations early, support isolation of their cause, and make recovery semantically meaningful. This chapter concerns pretraining health and reproducibility; it does not define downstream benchmark suites or post-training evaluation.

2 Training Loss, Validation Loss, and Perplexity

The training loss is the token-weighted Causal Language Modeling loss used to form gradients. Let $\mathcal{D}_{\text{train}}$ be the effective training distribution induced by the data mixture, packing policy, and loss mask, and let $\mathcal{D}_{\text{val}}$ be a fixed held-out evaluation distribution. For parameters θ_t at update t, write their population cross-entropies as

$$\mathcal{L}_{\text{train}(\theta_t)} = \mathrm{E}_{z\sim\mathcal{D}_{\text{train}}}[\ell(\theta_t; z)], \quad \mathcal{L}_{\text{val}(\theta_t)} = \mathrm{E}_{z\sim\mathcal{D}_{\text{val}}}[\ell(\theta_t; z)]. \tag{1}$$

The two quantities use the same token-level negative log-likelihood defined in Chapter 5, but they answer different questions. The training loss estimates the objective whose gradient changes θ_t. The validation loss estimates how that same probabilistic model scores examples withheld from the training stream. A validation set is not protected merely because it is stored in another file: it must be excluded from data mixture construction, deduplication decisions that would carry labels across the split, and any tuning procedure that would turn repeated evaluation into indirect training.

Neither loss is usually observed as the exact expectation in Equation 1. A training dashboard reports a finite, often short-window estimate whose variation includes sampling noise and changes in valid-token count. A validation event evaluates a finite, versioned slice under an explicit tokenizer, sequence

policy, and loss mask. The evaluated token count and reduction convention must accompany both values; otherwise a lower number may merely reflect different padding, truncation, or masking.

2.1 Perplexity and Evaluation Distributions

For a token-averaged validation loss measured with natural logarithms, validation perplexity is

$$\mathrm{PPL}_{\mathrm{val}} = \exp(\mathcal{L}_{\mathrm{val}}). \tag{2}$$

This is the same transform introduced in Chapter 5, now applied to a protected distribution. Perplexity makes a log-loss easier to compare within one fixed evaluation protocol, but it is not a tokenizer-independent measure and is not a task score. Changing the vocabulary, normalization, sequence boundary convention, or valid-token mask changes the event space being measured.

Validation perplexity also differs from downstream benchmark performance. It measures the likelihood assigned to observed continuation tokens under data prefixes; a downstream benchmark can require a different prompt distribution, an answer-extraction rule, a decoding policy, or a task-specific scoring function. Work on language-model evaluation illustrates that likelihood alone does not exhaust the behavioral properties one may want to measure [1]. A pretraining team should therefore use validation loss to monitor the declared objective while keeping downstream claims separate from it.

3 Evaluation Cadence and Online Metrics

Evaluation is not free. It consumes accelerator time, data-loader capacity, and attention from the operators who must interpret the result. Evaluating too rarely can let a bad trajectory consume a large token budget before it is recognized; evaluating too frequently can materially reduce training throughput or lead operators to overreact to small fluctuations. A practical cadence is defined in valid tokens or optimizer updates, records its own clock, and includes occasional larger evaluations in addition to lightweight frequent probes.

The online metrics surrounding a loss value provide its context. The learning rate and optimizer-step count explain the intended update scale. The consumed valid-token count connects a curve to Chapter 9′s token budget rather than to an implementation-dependent batch count. Throughput exposes stalls even when loss is healthy. Gradient norms, activation summaries, logit ranges, finite-value counts, and optimizer-state summaries expose failures before they necessarily appear as a divergent scalar loss. Chapter 8 explains why these reductions require a declared precision policy.

The point is not to log every tensor indiscriminately. A durable log separates fast scalar metrics from sampled distributional summaries. For example, record the training loss numerator and valid-token denominator, global gradient norm, learning rate, tokens per second, and finite-value flag at every update or a short interval. Record per-layer activation RMS, maxima or high quantiles, parameter norms, and moment statistics at a less frequent schedule or around an anomaly. Each series needs a stable name, unit, reduction rule, and association with the exact training state.

4 Diagnosing Spikes and Divergence

A loss spike is an observation, not a diagnosis. A **transient** spike is an unusually high loss that is followed by a return to the prior range while parameters, gradients, and optimizer state remain finite. **Sustained divergence** is a persistent deterioration, often accompanied by growing norms, repeated nonfinite values, stalled throughput, or a failure to recover after the next scheduled updates. The distinction matters because an automatic rollback for every outlying mini-batch can discard useful stochastic progress, while continuing through a corrupted update can make later evidence uninterpretable. Loss spikes have been documented as a practical problem in large-language-model pretraining; Takase et al. connect one important class to sudden gradient-norm growth [2].

The first task is to localize the earliest abnormal boundary. A data failure may be restricted to particular source records, malformed token sequences, unexpected loss masks, or a shifted mixture component.

An optimization failure can follow a learning-rate transition, a batch-size change, a warmup-clock error, or a growing gradient norm while arithmetic remains finite. A numerical failure is indicated by NaN or Inf values, loss-scale backoff, or strong sensitivity to widened arithmetic. A distributed failure can show as inconsistent rank-local counters, collective stalls, or an update taken by only part of a process group. A checkpoint failure can show only after restore, when missing state produces a discontinuity that the saved weights alone cannot explain.

Observed signal	First scope-preserving check	Failure class to investigate
One high but finite loss	Compare the exact batch, valid-token denominator, logit range, and learning-rate state with the preceding update.	Data anomaly, stochastic variation, or an optimization transition.
Repeated rising loss or norms	Hold the recipe fixed and inspect the first layer, parameter group, or schedule boundary that departs from baseline.	Optimization or architectural-scale instability.
NaN or Inf	Locate the first nonfinite tensor; rerun the preserved batch with selected reductions widened.	Numerical failure, invalid input, or a nonfinite update entering state.
Rank disagreement or stall	Compare rank-local counters, tensor shapes, and collective order before any restart.	Distributed execution or data-sharding fault.
Loss discontinuity after restore	Compare model, optimizer, scheduler, scaler, random, and sampler states against checkpoint metadata.	Incomplete or inconsistent checkpoint.

Table 1: A diagnostic table should preserve scope before changing a hyperparameter. The earliest abnormal boundary narrows a failure class; it does not establish a cause by itself.

Table 1 is deliberately ordered by tests that preserve the existing experiment. Lowering the learning rate, changing precision, or discarding a data source can be useful isolation experiments, but each also changes the trajectory. Preserve the failing batch identifiers, parameter revision, optimizer state, random state, data manifest, parallel configuration, and recent traces before applying a mitigation.

5 Checkpoints as Resumable Training States

A weight snapshot answers a narrow question: which parameters produced this model output? A resumable checkpoint answers a stronger question: which state is required for the next update to have the intended semantics? Let Ξ_t denote the optimizer state, σ_t the learning-rate scheduler state, s_t the gradient-scaler state when applicable, ρ_t the random-number-generator states, d_t the data-loader or sampler state, and π_t the distributed layout and sharding metadata. A conceptual checkpoint at update t is

$$\mathcal{C}_t = (\theta_t, \Xi_t, \sigma_t, s_t, \rho_t, d_t, \pi_t, t, N_{\text{consumed}}, \mu), \tag{3}$$

where N_{consumed} is the consumed valid-token count and μ records immutable run identity such as the model configuration, tokenizer version, data manifest, code revision, and precision policy. Some systems also record an in-progress gradient-accumulation state, pending asynchronous writes, or an evaluation cursor. The exact container varies, but the distinction in Equation 3 does not: model parameters are one component of the future computation.

The optimizer state matters because AdamW's first and second moments determine the next update, as Chapter 7 derives. Scheduler state matters because an update-indexed or token-indexed learning rate cannot be reconstructed safely from a vague training duration. The scaler state matters when dynamic loss scaling is active. Random states and sampler state matter because dropout, data order, augmentation, and worker initialization can change the subsequent gradient sequence. Restoring only θ_t can create a useful inference model, but it does not normally recreate the same training trajectory.

5.1 Full and Sharded Checkpoints

A **full** checkpoint materializes a logically complete state in a layout that can be read by one process or by a standard loading path. It is convenient for archival, interchange, or offline inspection, but gathering a large sharded model merely to write it can exceed memory or produce an expensive write bottleneck. A **sharded** checkpoint stores partitions across ranks or files, preserving the distributed placement needed for scalable I/O. It reduces gathering pressure, but its manifest must identify every shard, the model and optimizer mappings, the parallel layout, and the procedure for reconstructing or resharding the state.

Large-language-model checkpointing therefore trades storage, write interruption, recovery time, and implementation complexity. Let t_{save} be the training-visible cost of one checkpoint, let τ be a time interval between checkpoints, and let λ be an approximate failure rate per unit time. A simple steady-state accounting for the fractional overhead is

$$h(\tau) \approx \frac{t_{\mathrm{save}}}{\tau} + \frac{\lambda \tau}{2}. \tag{4}$$

The first term rewards less frequent saves; the second represents, in expectation, the work lost since the most recent save after a failure. This is a planning model, not a universal formula: failures are not always memoryless, restores have a cost, and an asynchronous save can overlap some of t_{save}. Its purpose is to make the trade-off explicit. LLM checkpointing systems must also account for the large, distributed model and optimizer state that turns a naive frequent write into a substantial I/O burden [3].

5.2 Resume Semantics and Checkpoint Validation

An **exact** resume is a demanding target: after reload, the next batch, masks, random draws, optimizer update, and counters agree with an uninterrupted run up to the stated numerical tolerance. This requires more than setting one global seed. It requires that data workers, rank-local random streams, sampler positions, accumulation boundaries, loss normalization, and distributed collectives be restored with compatible conventions. Bitwise identity can remain impossible across changed kernel versions, hardware, or collective-reduction order; this should be stated as a limitation rather than hidden as an unexplained drift.

A **semantic** resume is weaker but still meaningful. It preserves the model, optimizer, scheduler, data policy, and declared global counters even if a supported resharding operation changes the number of ranks or shard boundaries. The new run is not claimed to reproduce every random bit, but it remains an explicit continuation of the same objective. A change in world size is not automatically safe: the checkpoint format and loader must specify how parameter and optimizer shards map to the new layout. Modern distributed checkpoint interfaces provide such resharding mechanisms only when the state representation and metadata support them [4].

Checkpoint validation should occur before a failure makes it urgent. A writer should complete a manifest only after every required shard is durable and verifiable. A reader should check schema and version compatibility, file existence or checksums, tensor keys and shapes, finite values, optimizer parameter-group identity, and counter monotonicity. Periodically restore a recent checkpoint into an isolated process, run a small deterministic evaluation, and confirm that the measured loss and selected state summaries match the saved record. A checkpoint that has never been loaded successfully is a storage artifact, not a recovery plan.

6 Checkpoint Selection and Stopping Decisions

The checkpoint selected for an application need not be the most recent checkpoint. If the declared target is held-out Causal Language Modeling loss on a fixed validation distribution, select using a versioned validation protocol and retain the evaluation record with the artifact. If the target is a downstream capability, the selection criterion must be a separate task-level evaluation; it cannot be inferred from perplexity alone. Ranking checkpoints on a repeatedly inspected validation set also

creates selection pressure, so the final report should distinguish the development validation stream from any independently protected test or benchmark evaluation.

Classical early stopping halts training when validation performance no longer improves, using held-out data to avoid continuing into overfitting. Its criterion and patience window are themselves choices [5]. In large-scale pretraining, early stopping is less straightforward. The run may be planned around a fixed token or compute budget, validation loss may continue to improve slowly long after a narrow patience rule would trigger, evaluations can be expensive, and the available corpus may be far from exhausted. A stopped run may also be inferior under a later deployment objective even when it is locally best on one validation slice.

Early stopping is therefore a decision rule, not a default proof of generalization. It is most defensible when a protected metric has sustained deterioration, data reuse or distribution shift makes continued optimization harmful, or a controlled budget objective explicitly favors the best validation checkpoint. For a long planned pretraining run, validation should more often guide investigation and checkpoint retention than act as an unexamined stop signal.

7 Experiment Tracking and Reproducibility

Reproducibility begins with identity rather than with a random seed. Every measured curve should be associated with an immutable run configuration: architecture and parameter-count convention, tokenizer, source and data-manifest versions, mixture and packing policy, loss mask, optimizer and schedule, precision policy, distributed layout, code revision, hardware or runtime version where material, and evaluation datasets. The valid-token counter provides the common time axis connecting training loss, validation loss, learning rate, throughput, and checkpoints.

An experiment record should distinguish configuration from observation. Configuration specifies what the run intended to do. Observation records what it did: losses with numerators and denominators, gradient and activation summaries, learning rate, optimizer updates, consumed tokens, throughput, checkpoint identifiers, failures, restarts, and any manual intervention. This separation makes it possible to compare two runs that share a model name but not a data stream or schedule, and to diagnose why their outcomes differ.

Reproducibility is not an assertion that every implementation reproduces every low-order bit. It is the ability to reconstruct the declared computation closely enough to audit its trajectory, identify controlled differences, and resume it under stated semantics. That standard is stronger than archiving weights and more realistic than promising hardware-independent determinism.

8 Implementation Contracts

The metric contract must define the training and validation data identities, tokenizer version, sequence and masking policy, token-level loss numerator, valid-token denominator, evaluation cadence, and the counter that indexes each value. It must prevent evaluation examples from entering the training mixture and record any validation-set revision. Logged perplexity should be derived from the same natural-log token average as Equation 2, not from an incompatible sequence average.

The health contract should log learning rate, optimizer update, consumed valid-token count, throughput, loss, gradient norm, finite-value status, and checkpoint identifier under stable reduction rules. It should sample activation, logit, and optimizer-state statistics at a declared cadence, preserve enough recent history to analyze a spike, and make rank-local versus globally reduced metrics explicit. A skipped update or rollback must be recorded with its reason and agreed across the relevant distributed group.

The checkpoint contract must specify the complete state in Equation 3, whether the artifact is full or sharded, its manifest and integrity checks, the save-completion boundary, and its exact or semantic resume target. Test both a direct reload and a failure-style restore in which the process is recreated,

data is reinitialized, and the next optimizer step is compared against a controlled reference. Finally, link every selected checkpoint to the validation protocol and experiment record that justified it.

9 Summary

Training loss and validation loss measure related but different distributions: one drives updates, while the other estimates how the pretraining objective transfers to protected examples. Perplexity is a monotonic transform of token-averaged validation loss under a fixed protocol; it is not a substitute for task-specific evaluation. A reliable run therefore interprets loss together with token counts, learning rate, throughput, gradient and activation statistics, finite-value checks, and distributed health.

Checkpointing turns monitoring into recoverability only when the artifact contains the state that determines the next update. Parameters alone are not enough: optimizer and scheduler state, scaler state where applicable, random and sampler state, consumed-token count, configuration identity, and sharding metadata define whether a restart is exact or merely a new run from old weights. With validated checkpoints, protected evaluation, and explicit experiment records, training diagnostics become a disciplined way to preserve and investigate a pretraining trajectory rather than a collection of dashboard curves.

References

[1] C. Meister and R. Cotterell, "Language Model Evaluation Beyond Perplexity," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Association for Computational Linguistics, 2021, pp. 5328–5339. doi: 10.18653/v1/2021.acl-long.414.

[2] S. Takase, S. Kiyono, S. Kobayashi, and J. Suzuki, "Spike No More: Stabilizing the Pre-training of Large Language Models," *arXiv preprint arXiv:2312.16903*, 2023, doi: 10.48550/arXiv.2312.16903.

[3] A. Maurya, R. Underwood, M. M. Rafique, F. Cappello, and B. Nicolae, "DataStates-LLM: Lazy Asynchronous Checkpointing for Large Language Models," in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, 2024. doi: 10.1145/3625549.3658685.

[4] PyTorch Contributors, "Distributed Checkpoint." 2026. [Online]. Available: https://docs.pytorch.org/docs/stable/distributed.checkpoint.html

[5] L. Prechelt, "Early Stopping—but When?," in *Neural Networks: Tricks of the Trade*, vol. 1524, in Lecture Notes in Computer Science, vol. 1524., Springer, 1998, pp. 55–69. doi: 10.1007/3-540-49430-8_3.